

Day 19 : Provider State Management

1. Introduction to Provider

Provider is one of the most popular state management solutions in Flutter. It helps manage and share data efficiently across the entire application without passing data manually from one widget to another.

2. Why we use provider

- Makes code clean and organized.
- Separates UI from business logic.
- Reduces unnecessary widget rebuilding.
- Easier to maintain large applications.
- Improves app performance.

3. Provider vs setState()

Feature	setState()	Provider
Scope	Local widget only	Entire app
Code Structure	Simple but messy in large apps	Clean and scalable
Performance	Rebuilds whole widget	Rebuilds only required widgets
Suitable For	Small apps	Medium/Large apps
State Sharing	Difficult	Easy

4. Install Provider Package

Add dependency

```
dependencies:  
  flutter:  
    sdk: flutter  
  provider: ^6.1.5
```

5. Understanding ChangeNotifier

ChangeNotifier is a class that stores state and notifies widgets when data changes.

```
import 'package:flutter/material.dart';

class CounterProvider extends ChangeNotifier {
  int _count = 0;

  int get count => _count;

  void increment() {
    _count++;
    notifyListeners();
  }
}
```

notifyListeners()

This method tells all listening widgets to rebuild.

6. Setup Provider

Step 1: Wrap App with ChangeNotifierProvider

```
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => CounterProvider(),
      child: MyApp(),
    ),
  );
}
```

Step 2: Using MultiProvider

When using multiple providers:

```
void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(
          create: (_) => CounterProvider(),
        ),

        ChangeNotifierProvider(
          create: (_) => ThemeProvider(),
        ),
      ],
      child: MyApp(),
    ),
  );
}
```

7. Consuming State

A) Consumer Widget

```
Consumer<CounterProvider>(  
  builder: (context, provider, child) {  
    return Text(  
      provider.count.toString(),  
    );  
  },  
)
```

B) Provider.of<T>(context)

```
final provider =  
  Provider.of<CounterProvider>(context);  
  
Text(provider.count.toString());
```

C) context.watch<T>()

Listens for changes.

```
final provider =  
  context.watch<CounterProvider>();  
  
Text(provider.count.toString());
```

D) context.read<T>()

Used when listening is not required.

```
ElevatedButton(  
  onPressed: () {  
    context.read<CounterProvide  
      .increment();  
  },  
  child: Text("Increment"),  
)
```

8. Provider Types

1. Provider

Provides simple values.

```
Provider<String>(  
  create: (_) => "Flutter",  
)
```

2. ChangeNotifierProvider

Provides ChangeNotifier classes.

```
ChangeNotifierProvider(  
  create: (_) => CounterProvider(),  
)
```

3. FutureProvider

Provides future data.

```
FutureProvider<int>(  
  create: (_) async => 100,  
  initialData: 0,  
)
```

4. StreamProvider

Provides stream data.

```
StreamProvider<int>(  
  create: (_) => numberStream(),  
  initialData: 0,  
)
```

5. ProxyProvider

Depends on another provider.

```
ProxyProvider<AuthProvider, CartProvider>(  
  update: (_, auth, cart) =>  
    CartProvider(auth.token),  
)
```

9. Authentication State with Provider

```
class AuthProvider extends ChangeNotifier {  
  
  bool _isLoggedIn = false;  
  
  bool get isLoggedIn => _isLoggedIn;  
  
  void login() {  
    _isLoggedIn = true;  
    notifyListeners();  
  }  
  
  void logout() {  
    _isLoggedIn = false;  
    notifyListeners();  
  }  
}
```

10. Usage

```
if (provider.isLoggedIn) {  
  return HomeScreen();  
} else {  
  return LoginScreen();  
}
```

11. Multiple Providers Example

```
MultiProvider(  
  providers: [  
  
    ChangeNotifierProvider(  
      create: (_) => AuthProvider(),  
    ),  
  
    ChangeNotifierProvider(  
      create: (_) => CartProvider(),  
    ),  
  
    ChangeNotifierProvider(  
      create: (_) => ThemeProvider(),  
    ),  
  
  ],  
  child: MyApp(),  
)
```

Getting API Key :

Registration complete

Your API key is: `8a24150527d047cda99826321c6ad863`

For help getting started please look at our [getting started guide](#).

We post API status updates and other news on our Twitter feed, so please follow us there if that's important to you:

 Follow @NewsAPIorg

 My account

